

Agrovision.Ai: Easy-to-Understand Viva & Study Guide

A simple, detailed, and beginner-friendly breakdown of your project.

1. The Big Picture: What Does This Project Actually Do?

Imagine a poor farmer who needs expert advice but cannot afford to hire an agricultural scientist or wait weeks for laboratory soil tests. **Agrovision.Ai acts as a virtual scientist right in their pocket.**

Here is exactly how it works, step-by-step in simple terms:

- **Step 1: The Input (What the user does)**

The farmer simply uploads a photo of their farm or a sick leaf and clicks "Share Location". That's it! No long, complicated forms to fill out.

- **Step 2: Gathering the Missing Pieces (Behind the scenes)**

Our system takes that GPS location and automatically asks the internet for more details. It asks a weather service (Open-Meteo) "*What is the weather like here?*" and asks a global soil database (SoilGrids) "*What chemicals are in the soil at this exact spot?*".

- **Step 3: The Custom AI Analysis**

The uploaded photo is first sent through our **Personally Trained Machine Learning Model**. We specifically trained this model to recognize local crop diseases and analyze farm health.

- **Step 4: The Final Output**

The results from our custom model are combined with the weather and soil data, and finalized by Google Gemini to create a simple, easy-to-read plan: What crop to grow, when to water it, what fertilizer to use, and how to cure the

disease. Our system turns this into a beautiful PDF report and emails it to the farmer.

- **Step 5: Voice Assistant**

If the farmer cannot read well, they can click a microphone icon and ask questions in English, Hindi, or Marathi, and the website will actually speak the answers out loud!

2. Technologies Used (And Why We Chose Them)

Custom Trained Machine Learning Model (The Specialized Farm Expert)

What it is: A custom-built AI (like a Convolutional Neural Network using TensorFlow/PyTorch) that we trained ourselves using thousands of pictures of local farms and diseased leaves.

Why we used it: Standard, out-of-the-box AI is sometimes too generic. By training our own model, we ensured the system acts as a highly specialized expert capable of detecting specific local diseases much faster and more accurately.

Next.js & React (The Face & Nervous System)

What it is: React is used to build the buttons and screens you see. Next.js is a tool that makes React faster and allows us to write backend code (like API routes) in the exact same place.

Why we used it: It makes the website extremely fast and easy to build without needing to set up a completely separate server for the backend.

Supabase (The Secure Filing Cabinet)

What it is: Supabase is a cloud database service (based on a popular database called PostgreSQL).

Why we used it: We need a place to safely store user accounts, passwords, and their generated reports. Supabase provides a feature called "Row-Level Security", which acts like a bouncer, ensuring Farmer A can never accidentally see Farmer B's private reports.

Open-Meteo & SoilGrids APIs (The Automated Reporters)

What it is: Free web services that provide live data to developers.

Why we used it: To save the farmer's time. Instead of forcing the farmer to manually type in their local temperature or take their soil to a lab to find the pH level, these APIs fetch that data instantly over the internet using just the GPS coordinates.

3. Common Examiner Questions & Easy Answers

Q1: You mentioned using a custom-trained model. How exactly did you train it?

Easy Answer: We collected a large dataset of farm and plant leaf images (like from Kaggle or PlantVillage) that represented the specific diseases we wanted to target. We labeled these images and used a framework like TensorFlow (or PyTorch) to train a Convolutional Neural Network (CNN). The model learned to recognize patterns, like brown spots or yellowing edges, to diagnose the plant's health.

Q2: Why did you train your own model instead of just using Gemini or ChatGPT for everything?

Easy Answer: General models like Gemini are great for chatting and basic analysis, but they aren't agricultural specialists. By training our own model specifically on plant diseases, we got much higher accuracy and faster response times for image detection. We then only use Gemini at the very end to format our custom model's results into human-readable advice.

Q3: How does your system fetch Weather, Soil, and Location data so quickly? Why doesn't it freeze?

Easy Answer: If you go to a restaurant, you don't order a burger, wait for it to arrive, then order fries, wait, and then order a drink. You order them all at once! In our code, we use something called `Promise.all()`. This tells the computer to fetch the weather, the soil, and the location all at the exact same time (in parallel) rather than one by one. This cuts the loading time by a third!

Q4: Why did you use PostgreSQL (Supabase) instead of a NoSQL database like MongoDB?

Easy Answer: Agricultural data is highly connected. A "User" owns a "Farm", and a "Farm" has many "Reports". Relational databases like PostgreSQL are specifically designed to link these tables together cleanly and securely, whereas NoSQL is better for random, unconnected data.

Q5: What if the Weather API is down or broken? Will the whole app crash?

Easy Answer: Our code uses try-catch blocks, which act like safety nets. If the weather service is broken, the safety net catches the error and simply sends "Weather: Unknown" to our system instead of crashing the whole app. The system is smart enough to still give advice based on the photo and soil data alone.

Q6: How did you make the Voice Assistant speak in regional languages like Marathi? Did you pay for a premium service?

Easy Answer: No, we didn't pay for an external service! We used the **Web Speech API**, which is a hidden feature already built into modern browsers like Google Chrome. It uses the phone's own built-in tools to listen to the farmer (Speech-to-Text) and then speak the answers back out loud (Text-to-Speech) in Hindi or Marathi.

Q7: What is the main real-world problem this project solves?

Easy Answer: It brings high-tech science to poor or uneducated farmers. Normally, precision farming is only for giant, wealthy corporations. By making an app that requires zero typing—just a photo and a click—we give the poorest farmer the exact same expert advice that a rich corporation has, completely for free.